14. ROS 2 DDS

1. Introduction to DDS

DDS stands for Data Distribution Service. It was released and maintained by the Object Management Group (OMG) in 2004. It is a set of data distribution/subscription standards designed specifically for real-time systems. It was first applied by the US Navy to solve the compatibility problem of a large number of software upgrades in the complex network environment of ships, and it has now become a mandatory standard.

DDS emphasizes being data-centric and can provide rich quality of service policies to ensure real-time, efficient, and flexible data distribution, meeting the needs of various distributed real-time communication applications.

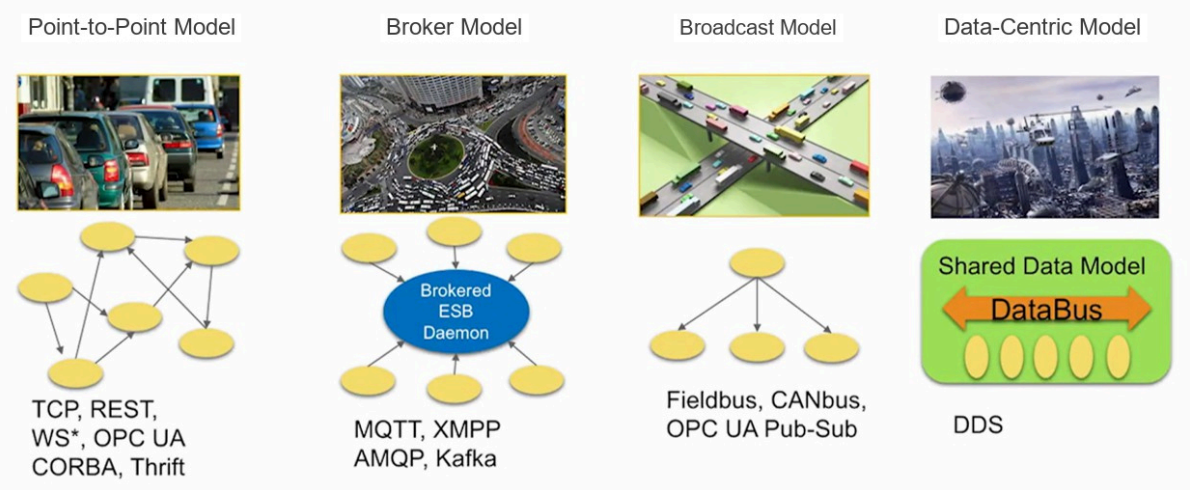
References:

- Fast DDS official website documentation: 3.3.0
- ROS 2 official documentation on advanced DDS features: Click to jump

2. Communication Model

The topics, services, and actions we learned in the previous courses all rely on DDS for the specific implementation process of their underlying communication. It is equivalent to the neural network in the ROS robot system.

The core of DDS is communication. There are very many models and software frameworks that can implement communication. Here we list four commonly used models.



- The first is the **point-to-point model**. Many clients connect to one server, and each communication requires the two communicating parties to establish a connection. When the number of communicating nodes increases, the number of connections also increases. Moreover, each client needs to know the specific address of the server and the services it provides. Once the server address changes, all clients are affected.
- The second is the **Broker model**, which is optimized for the point-to-point model. The Broker centrally processes everyone's requests and further finds the roles that can truly respond to the service. In this way, the client does not need to care about the specific address of the server. However, the problem is also obvious: as the core, the processing speed of the Broker affects the efficiency of all nodes. When the system scale grows to a certain extent, the Broker will become the performance bottleneck of the entire system. What is more

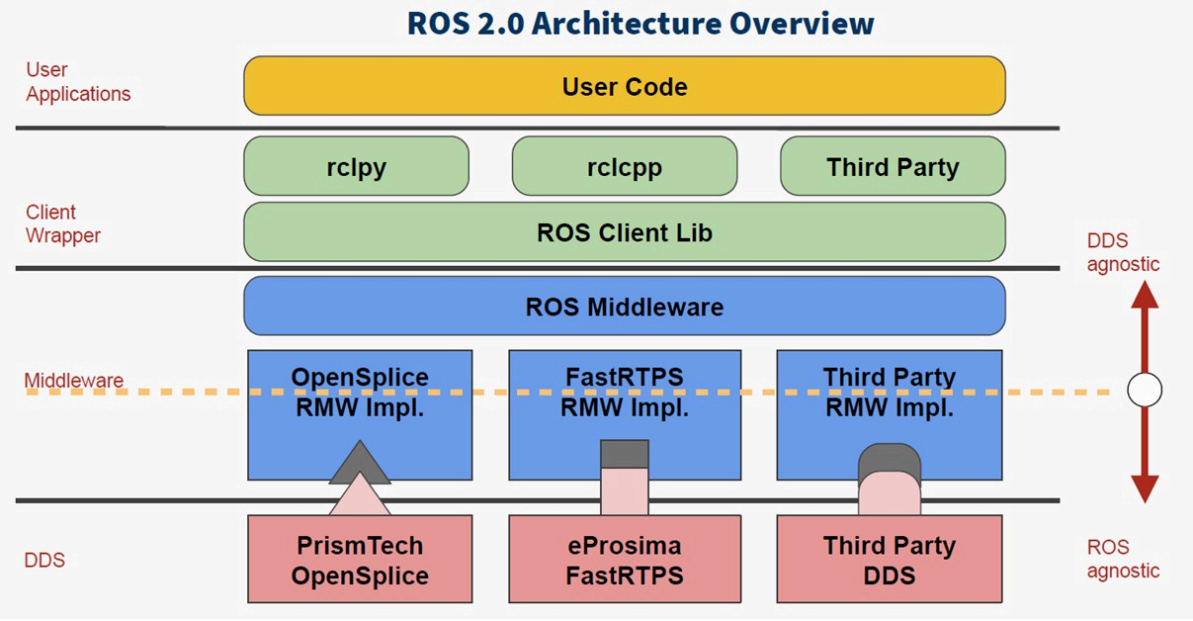
troublesome is that if an exception occurs in the Broker, the entire system may not be able to operate normally. The previous ROS1 system used an architecture similar to this.

- The third is the **broadcast model**. All nodes can broadcast messages on the channel, and all nodes can receive the messages. This model solves the problem of the server address, and the two communicating parties do not need to establish a separate connection. However, there are too many messages on the broadcast channel, and all nodes must care about every message, while in fact many of them have nothing to do with themselves.
- The fourth is the **data-centric DDS model**. This model is somewhat similar to the broadcast model: all nodes can publish and subscribe to messages on the DataBus. But its advanced point is that the communication contains many parallel pathways, and each node can only care about the messages it is interested in and ignore the messages it is not interested in. It is somewhat like a rotating hot pot, where all kinds of delicious food are transmitted on this DataBus, and we only need to take what we want to eat; the rest has nothing to do with us.

It can be seen that among these several communication models, the advantages of DDS are more obvious.

3. Application of DDS in ROS 2

The position of DDS in the ROS 2 system is crucial, and all upper-layer construction is built on top of DDS. In this architecture diagram of ROS 2, the blue and red parts are DDS.



Among the four major components of ROS, due to the addition of DDS, the comprehensive capability of the distributed communication system is greatly improved. In this way, in the process of developing robots, we do not need to struggle with communication problems and can spend more time on application development of other parts.

4. Quality of Service Policy QoS

The basic structure in DDS is the Domain. The Domain binds various applications together for communication. Recall that when we configured communication between the Raspberry Pi and the computer before, the DOMAIN ID configured at that time is the grouping definition of the global data space. Only nodes in the same DOMAIN group can communicate with each other. This avoids resources being occupied by useless data.

Another important feature in DDS is the quality of service policy: QoS.

QoS is a network transmission policy. The application specifies the required network transmission quality behavior, and the QoS service implements the requirements of this behavior, satisfying the customer's demand for communication quality as much as possible. It can be understood as a contract between the data provider and the receiver.

The policies are as follows:

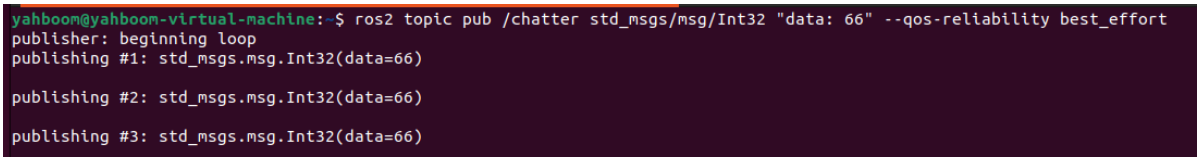
- **DEADLINE** policy, which means that communication data must complete one communication within each deadline;
- **HISTORY** policy, which means a cache size for historical data;
- **RELIABILITY** policy, which means the mode of data communication. If it is configured as BEST_EFFORT, it is the best-effort transmission mode: data flow must also be guaranteed when the network condition is poor, and data loss may occur at this time. If it is configured as RELIABLE, it is the reliable mode, which can ensure the integrity of the image as much as possible during communication. We can select the appropriate communication mode according to the application function scenario;
- **DURABILITY** policy, which can be configured so that a certain amount of historical data is also guaranteed to be sent to nodes that join late, allowing new nodes to quickly adapt to the system.

5. Test Case

5.1. Case 1—Configuring DDS via the Command Line

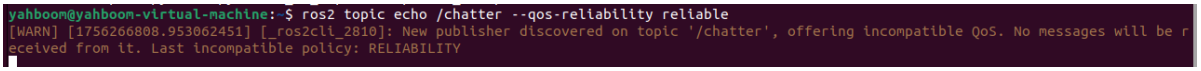
- Open the first terminal and publish a topic with the following command:

```
ros2 topic pub /chatter std_msgs/msg/Int32 "data: 66" --qos-reliability
best_effort
```



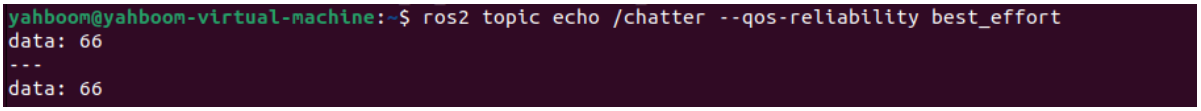
- Open another terminal and print the topic with a different QoS. If the QoS policy used is different from that of the publisher, a warning message will appear and the topic data cannot be received normally:

```
ros2 topic echo /chatter --qos-reliability reliable
```



- If we use the same QoS policy as the topic publisher, the topic data can be received

```
ros2 topic echo /chatter --qos-reliability best_effort
```



5.2. Case 2—Writing Topic Nodes to Configure QoS Service Policies

- Create a new package with the following command

```
ros2 pkg create learning_dds --build-type ament_python --dependencies rclpy
std_msgs
```

- Create a new dds_controller_pub.py file as the publisher of topic communication, and fill in the following content:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
# Import QoS-related classes
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy

class ControllerPublisher(Node):
    def __init__(self, name):
        super().__init__(name)
        # 1. Configure the QoS policy: reliable transmission, keep the last 1
piece of historical data
        self.qos_profile = QoSProfile(
            reliability=QoSReliabilityPolicy.RELIABLE,  # Reliable transmission
(retransmit lost data)
            history=QoSHistoryPolicy.KEEP_LAST,         # Keep the last N pieces
of data
            depth=1                                     # Keep 1 piece of
historical data
        )
        # 2. Create a publisher: topic name /robot_cmd, message type String, QoS
policy
        self.publisher = self.create_publisher(
            String,
            "/robot_cmd",
            self.qos_profile
        )
        # 3. Create a timer: send a command once per second
        self.timer = self.create_timer(1.0, self.timer_callback)
        self.cmd_list = ["forward", "backward", "stop"]  # Command list
        self.cmd_index = 0  # Command index, switched in a loop


    def timer_callback(self):
        # Switch commands in a loop (forward -> backward -> stop -> forward...)
        current_cmd = self.cmd_list[self.cmd_index % 3]
        # Create a message and fill in the data
        msg = String()
        msg.data = current_cmd
        # Publish the message
        self.publisher.publish(msg)
        # Print the log (display the published command)
        self.get_logger().info(f"Publishing control command: {msg.data}")
        # Update the command index
```

```
        self.cmd_index += 1


def main(args=None):
    # Initialize ROS 2
    rclpy.init(args=args)
    # Create the publisher node
    node = ControllerPublisher("robot_controller_pub")
    # Run the node in a loop
    rclpy.spin(node)
    # Destroy the node and shut down ROS 2
    node.destroy_node()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

- Implementation of the subscriber code: create a new dds_robot_sub.py file and fill in the subscriber implementation code below

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy

class RobotSubscriber(Node):
    def __init__(self, name):
        super().__init__(name)
        # 1. Configure a QoS policy compatible with the publisher
        self.qos_profile = QoSProfile(
            reliability=QoSReliabilityPolicy.BEST_EFFORT,
            history=QoSHistoryPolicy.KEEP_LAST,
            depth=1
        )
        # 2. Create a subscriber: topic name /robot_cmd, callback function, QoS
policy
        self.subscription = self.create_subscription(
            String,
            "/robot_cmd",
            self.cmd_callback,  # The callback function executed after receiving
data
            self.qos_profile
        )

    def cmd_callback(self, msg):
        # Callback function: process the received command
        self.get_logger().info(f"Received control command: {msg.data} -> Execute
the corresponding action")

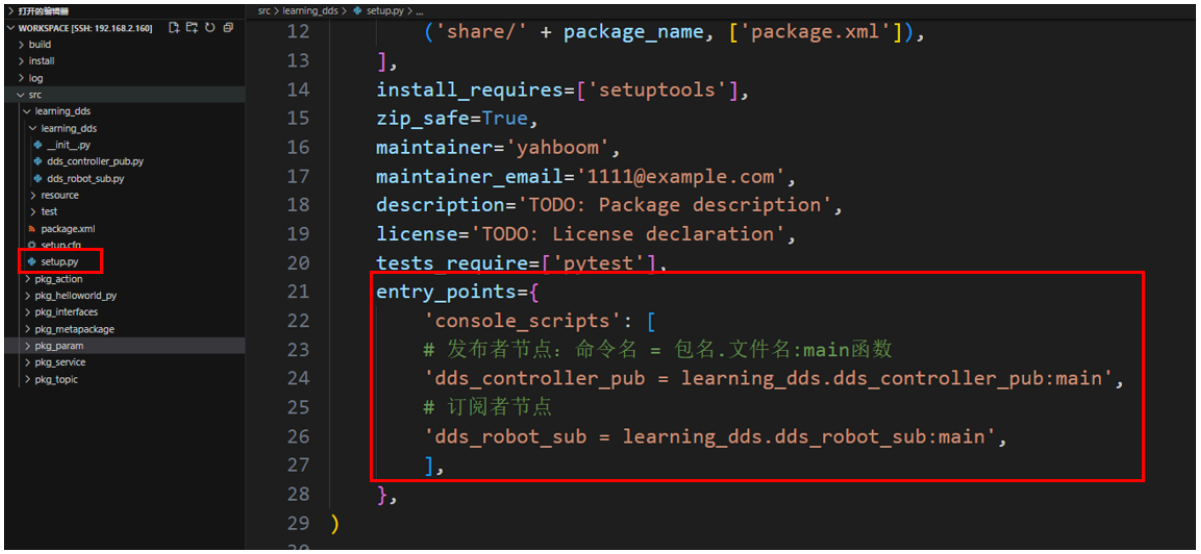

def main(args=None):
    rclpy.init(args=args)
    node = RobotSubscriber("robot_subscriber")
    rclpy.spin(node)
```

```
    node.destroy_node()
    rclpy.shutdown()


if __name__ == "__main__":
    main()
```

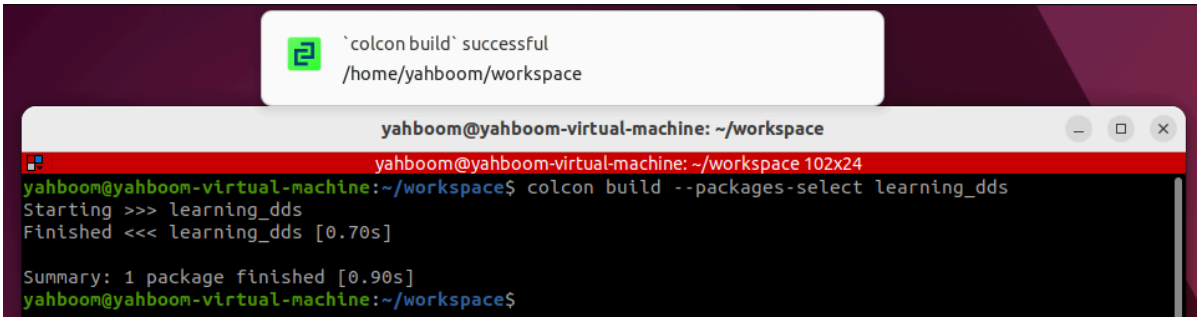
- Configure the compilation file (setup.py)

```
entry_points={
    'console_scripts': [
        # Publisher node: command name = package name.file name:main function
        'dds_controller_pub = learning_dds.dds_controller_pub:main',
        # Subscriber node
        'dds_robot_sub = learning_dds.dds_robot_sub:main',
    ],
},
```



- Open a terminal in the workspace directory and compile the package

```
colcon build --packages-select learning_dds
```



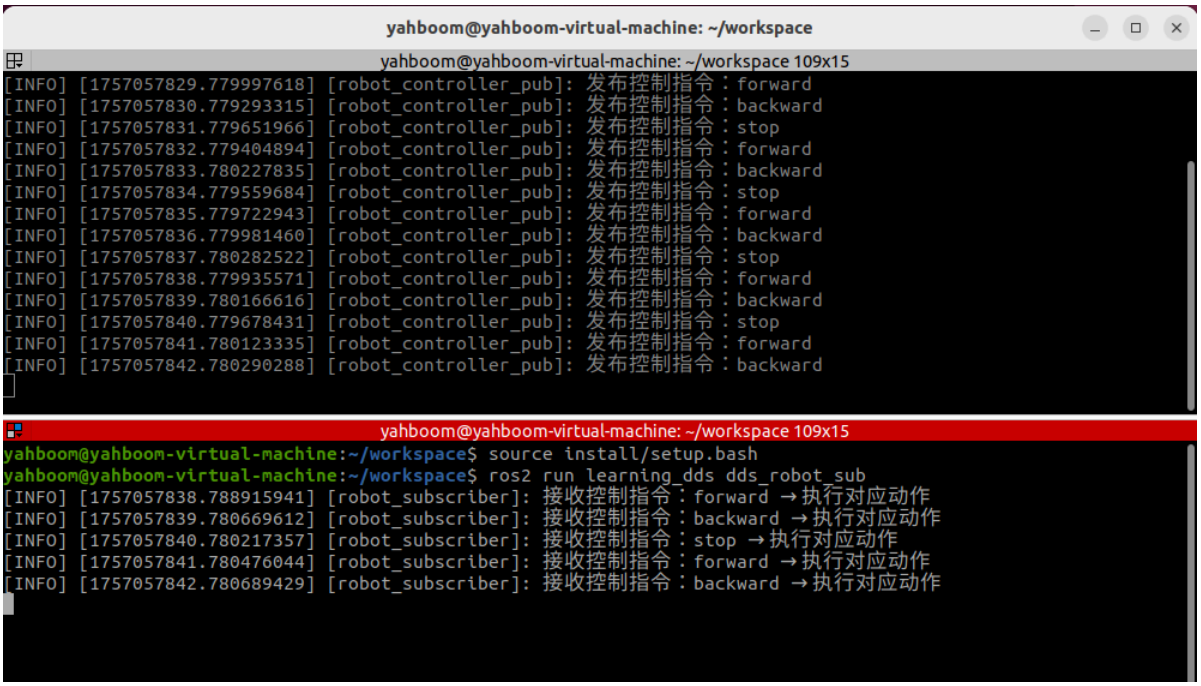
- Set the environment variables. The environment variables need to be set after every recompilation

```
source install/setup.bash
```

- Run the publisher and subscriber nodes

```
ros2 run learning_dds dds_controller_pub
```

```
ros2 run learning_dds dds_robot_sub
```



Advanced Robot Development:

- Usually, the publisher and subscriber of topic communication need to maintain the same QoS communication policy to avoid some hidden communication layer problems;
- The setting of the QoS communication policy should be selected according to the actual application scenario;
- If the application scenario involves drone communication, encrypted communication, real-time communication, and other scenarios that have special requirements for the communication layer, the official Fast-DDS manual can be found in the introduction of this tutorial, which has more detailed descriptions of DDS functions;

